

Contenu

Contexte	2
Qu'est-ce qu'une procédure stockée	3
Syntaxe générale d'une procédure stockée	3
Procédure stockée sans paramètre	4
Application 1 : Liste des locations non encore réglées	4
Procédure stockée avec paramètres d'entrée.....	5
Application 2 : liste des locations d'un client donné.....	5
Application 3 : Enregistrement du règlement d'une location.....	6
Procédure stockée avec paramètres de sortie	7
Application 4 : Prix d'un studio en fonction de la saison	7
La gestion des erreurs : le Try... Catch.....	8

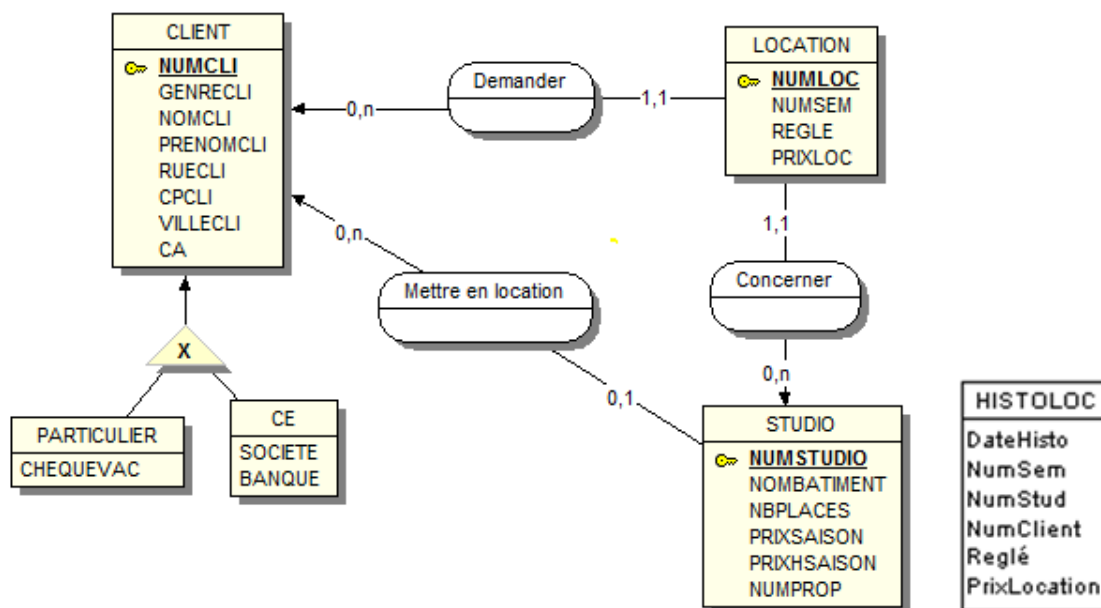
Contexte

L'agence immobilière **Mer et Soleil** propose la location d'appartements sur le Languedoc Roussillon. Elle dispose de différentes agences situées sur Montpellier, La Grande Motte et Agde. Elle souhaite mettre en place une application base de données répartie permettant aux différentes agences de gérer les locations d'appartements.

Suite à l'analyse effectuée, les principales **règles de gestion** suivantes ont été formalisées :

- RG1** On distingue deux types de client : les particuliers et les comités d'entreprise (CE). La notion de client inclut la notion de propriétaire.
- RG2** Le chiffre d'affaires (CA) global réalisé est enregistré dans la fiche du client.
- RG3** Un studio doit être mis en location par un client propriétaire avant d'être loué.
- RG4** Les locations se font à la semaine et comence le samedi.
- RG5** La saison haute va du 01/05 au 15/09.
- RG6** Toute semaine commencée en saison haute est considérée comme haute.
- RG7** Les particuliers peuvent régler à l'aide de chèque vacances.
- RG8** Pour les comités d'entreprise, on conserve le nom de la société et de la banque.
- RG9** En fin d'année, les locations supprimées sont historisées.

Une étude a permis d'établir le **modèle conceptuel de données** suivant :



Le **modèle relationnel** correspondant est :

- CLIENT (NumCli, GenreCli, NomCli, PrenomCli, RueCli, CPcli, VilleCli, CA)
- PARTICULIER (NumCli, ChequeVac)
- CE (NumCli, NomSociete, Banque)
- STUDIO (NumStu, NomBat, NbPlaces, PrixSaison, PrixHSaison, #NumProp)
- LOCATION (NumLoc, #NumStu, #NumCli, NumSem, Reglé, PrixLoc)

- HISTOLOC (id, DateHisto, NumSem, NumStud, NumClt, Reglé, PrixLoc)

Qu'est-ce qu'une procédure stockée

Une **procédure stockée (PS)** est un ensemble d'**instructions Transact-SQL** qui réalise des traitements (requêtes) sur une base de données.

Lorsqu'une application doit **exécuter une requête** dans une base de données, il est préférable que celle-ci appelle une **procédure stockée** plutôt que d'envoyer une commande T-SQL construite dans le code de l'application.

Les **avantages** sont nombreux :

- **Sécurité** contre les attaques par injection
- **Maintenabilité** du code par une programmation modulaire et réutilisable (découpage du code, passage de paramètres...)
- **Réduction du trafic** réseau entre les applications et le serveur
- **Compilation** de la PS en un **plan unique** qui demeure dans le cache de procédures, ce qui réduit considérablement le temps de travail du processeur.

Le langage **Transact-SQL** propose une **programmation procédurale** associée à du **langage SQL**.

Une **PS** est exécutée sur le **serveur**.

Une PS peut :

- Accepter des **paramètres d'entrée**
- Utiliser des **variables locales**
- **Renvoyer des données** (paramètres de sortie, codes de retour, résultats d'instructions SELECT, curseurs)

On distingue **3 types** de PS :

- PS **systèmes** (préfixe sp_) : gestion de SQL Server et affichage d'informations sur les données et les utilisateurs
- PS **étendues** (préfixe xp_) : bibliothèques de DLL.
- PS **utilisateur**

Une PS est composée de code Transact-SQL.

Syntaxe générale d'une procédure stockée

```
CREATE PROCEDURE Nom_PS[(parametres)]
AS
    BEGIN
        Instructions
    END
GO
```

Pour modifier une PS, remplacer **CREATE** par **ALTER**.

Procédure stockée sans paramètre

Application 1 : Liste des locations non encore réglées

Cette PS liste toutes les locations non réglées.

a) Création de la PS :

- Ouvrir la base de données sous SQL Server Management Studio
- Programmabilité / Procédures stockées / Nouvelle PS
- Action / Nouvelle PS
- Saisir le code de la PS, la tester et l'exécuter

```
CREATE PROCEDURE LocNonReglee
AS
    SELECT NomCli, NumStu, NumSem, NumStudio, NomBatiment
    FROM Client, Location, Studio
    WHERE Client.NumCli = Location.NumCli
    AND Studio.NumStudio = Location.NumStu
    AND Location.Regle = 0
GO
```

- Tester la PS et l'exécuter. Si elle ne comporte pas d'erreur, elle est visible dans la liste des PS.

b) Appel et utilisation de la PS :

IMPORTANT : Avant de tester votre PS, vérifier les données présentes dans les tables Studio, Client et surtout Location. Dans la table Location, le champ « Regle » doit être à False, pour avoir des locations non réglées.

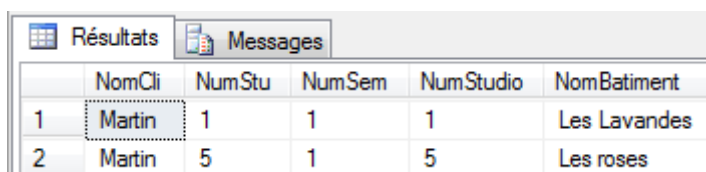
Solution 1 : clic droit sur la PS et exécuter

Solution 2 : par le code

- Créer une nouvelle requête
- Saisir le code d'appel

```
EXEC LocNonReglee
```

- Le résultat apparait en fonction des données des tables



	NomCli	NumStu	NumSem	NumStudio	NomBatiment
1	Martin	1	1	1	Les Lavandes
2	Martin	5	1	5	Les roses

Procédure stockée avec paramètres d'entrée

Une PS peut comporter jusqu'à 1024 paramètres.

Pour spécifier un paramètre, faire précéder son identificateur de @.

Application 2 : liste des locations d'un client donné

Cette PS liste toutes les locations d'un client dont le nom est saisi en paramètre d'entrée.

a) Création de la PS :

```
CREATE PROCEDURE LocClient @NomClient char(50)
AS
    SELECT NumStu, NumSem, NumStudio
    FROM Client, Location, Studio
    WHERE Client.NumCli = Location.NumCli
    AND Studio.NumStudio = Location.NumStu
    AND Client.NumCli = @NomClient
```

NB : Pour affecter une valeur par défaut au paramètre, modifier l'en tête comme suit :

```
-- Le nom client est passé en paramètre d'entrée (A défaut, Legrand sera utilisé)
CREATE PROCEDURE LocClient @NomClient char(50) = 'Dupont'
```

b) Appel et utilisation de la PS :

IMPORTANT : Avant de tester votre PS, vérifier les données présentes dans les tables concernées.

```
EXEC LocClient 'DUPONT'
```

- Le résultat apparait en fonction des données des tables

	NumStu	NumSem	NumStudio
1	2	2	2
2	3	4	3

Notes personnelles

Application 3 : Enregistrement du règlement d'une location

Cette PS enregistre le fait qu'une location soit réglée (le champ **Regle** passe à 1). Elle reçoit en paramètres d'entrées le numéro du studio et le numéro de la semaine.

a) Création de la PS :

```
CREATE PROCEDURE EnregReglement(@NumStudio int, @NumSemaine int)
AS
    BEGIN
        UPDATE Location
        SET Regle=1
        WHERE NumStu = @NumStudio
        AND NumSem = @NumSemaine;
    END;
```

b) Appel et utilisation de la PS :

Elle passe à 1 (TRUE) le champ Regle pour le studio n°1 en semaine n°1

```
EXEC EnregReglement 1,1
```

Notes personnelles

Procédure stockée avec paramètres de sortie

Application 4 : Prix d'un studio en fonction de la saison

Cette PS renvoie le prix d'un studio en saison et hors saison. Elle reçoit en paramètres d'entrées un numéro de studio.

a) Création de la PS :

```
-- Cette PS affiche les prix saison et hors saison d'un studio dont le n° est passé en
paramètres
CREATE PROCEDURE PrixLocStudio @NumStudio int,                -- param entrée
                                @PrixStudioS money OUTPUT,    -- param sortie 1
                                @PrixStudioHS money OUTPUT    -- param sortie 2
AS
BEGIN
    SELECT @PrixStudioS=PrixSaison, @PrixStudioHS=PrixHSaison
    FROM Studio
    WHERE NumStudio = @NumStudio
END
```

b) Appel et utilisation de la PS :

```
-- Exécution de la PS
DECLARE @Prix1 float
DECLARE @Prix2 float

EXECUTE PrixLocStudio 2, @PrixStudioS=@Prix1 OUTPUT, @PrixStudioHS=@Prix2 OUTPUT

PRINT @Prix1
PRINT @Prix2
```

- Le résultat apparaît en fonction des données des tables

	Résultats	Messages
	@PrixStudioS	@PrixStudioHS
1	500,00	400,00

Notes personnelles

La gestion des erreurs : le Try... Catch

La **gestion des erreurs** est assurée par l'encadrement du code par un bloc **Try... Catch** qui intercepte les erreurs qui peuvent apparaître dans les instructions présentes dans le Try et passe le script dans une partie prévue à cet effet.

Application : Dans l'exemple ci-dessous, une procédure tente d'enregistrer la location d'un client en donnant le n° client, son nom, son prénom, le n° du studio et le n° de la semaine.

a) Création de la PS :

```
CREATE PROCEDURE [dbo].[EnregLocationClient] @NomClient varchar(30),
                                             @PrenomClient varchar(30),
                                             @NumStu int,
                                             @NumSem int
AS
BEGIN TRY
    INSERT INTO Client (NomCli, PrenomCli)
    VALUES (@NomClient, @PrenomClient)

    INSERT INTO Location (NumStu, NumSem, NumCli)
    VALUES (@NumStu, @NumSem, @@IDENTITY)
END TRY

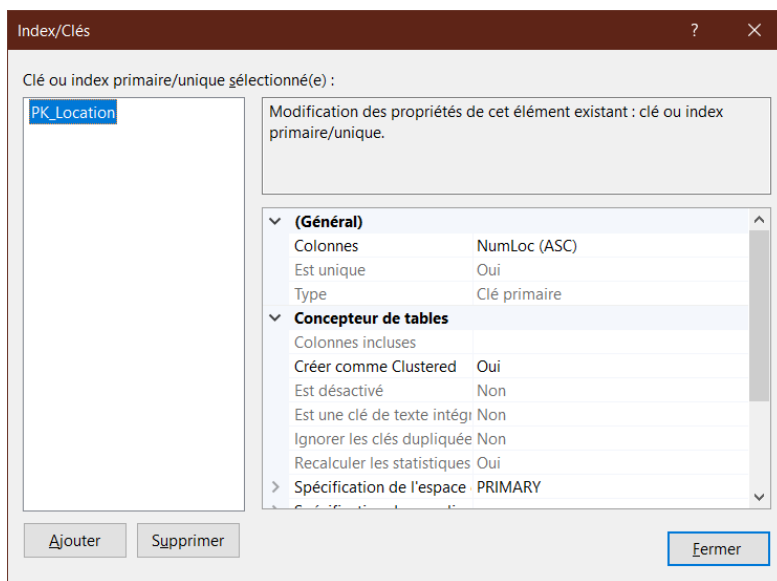
BEGIN CATCH
    SELECT ERROR_NUMBER() AS NumeroErreur,
           ERROR_SEVERITY() AS SeveriteErreur,
           ERROR_STATE() AS EtatErreur,
           ERROR_PROCEDURE() AS ProcedureErreur,
           ERROR_LINE() AS LigneErreur,
           ERROR_MESSAGE() AS MessageErreur;
END CATCH;
```


b) Analyse de la codification de la table Location :

- Un problème se pose sur les données de la table Location.
- La clé de la table Location est uniquement : NumLoc.
- Or un studio, une semaine donnée ne peut pas être loué plusieurs fois.
- Donc il faut créer un index unique sur les champs NumSem, NumStu dans Location

Procédure :

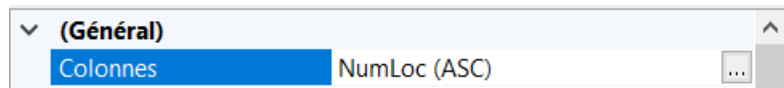
1. Ouvrir la table **Location** en **mode Création**
2. Cliquer sur l'icône **Gérer les index et les clés :**



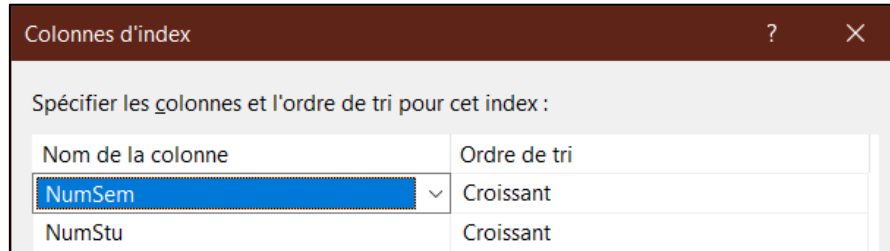
La clé primaire, index unique, apparaît.

3. Cliquer sur **Ajouter**
4. Passer (dans Général) **Est unique** à **Oui**

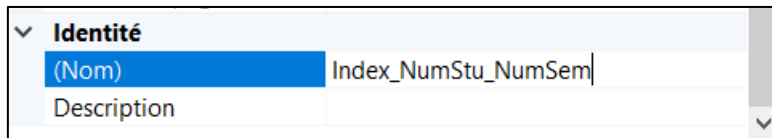
5. Cliquer sur les ... en fin de Colonnes



et sélectionner les 2 champs à indexer :



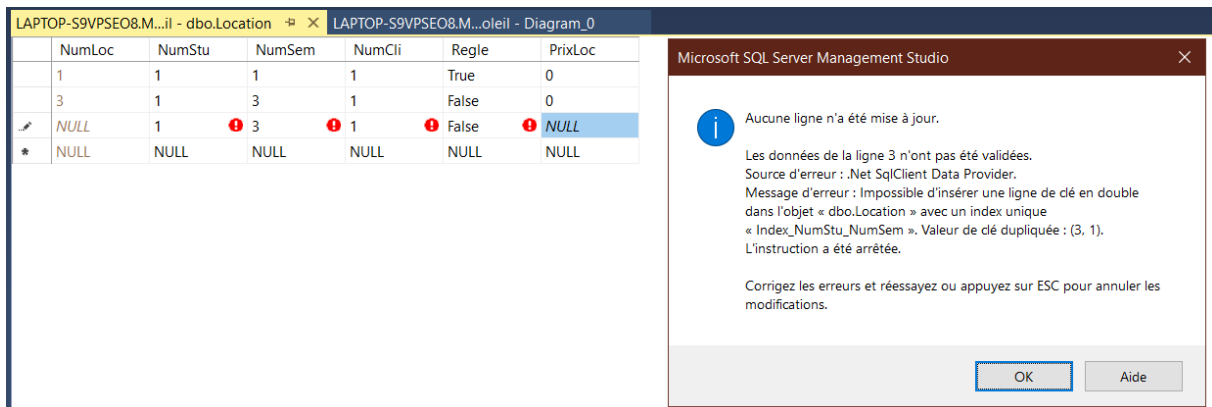
6. Nommer l'index :



7. Tester le fonctionnement de l'index :

Pour cela :

- Ouvrir la table Location (Modifier les 200 lignes)
- Tenter de saisir 2 lignes avec le même studio sur la même semaine (dans l'exemple ci-dessous le studio 1 la semaine 3)



c) Appel et utilisation de la PS :

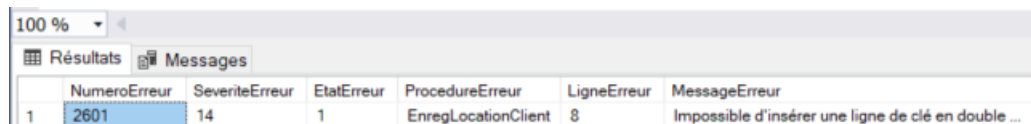
Exec EnregLocationClient "Martinez", "Loic", 1, 1

Le N° de client (**NumCli**), de type **Compteur**, est créé automatiquement par SQL Server.

Remarques importantes :

- La variable **@@IDENTITY** permet de récupérer cette valeur pour l'insérer en clé étrangère dans la table **Location**
- Toute erreur sera gérée par le TRY CATCH :

Par exemple, en appelant 2 fois la PS ci-dessus, le studio 1 serait loué deux fois en semaine 1. Ainsi, le traitement en erreur du **TRY** est basculé vers le block **CATCH**.



The screenshot shows the 'Messages' tab in SQL Server Enterprise Manager. It displays a single error message with the following details:

	NumeroErreur	SeveriteErreur	EtatErreur	ProcédureErreur	LigneErreur	MessageErreur
1	2601	14	1	EnregLocationClient	8	Impossible d'insérer une ligne de clé en double ...